2.4 How do forms with PATCH, PUT, or DELETE methods work?

The Rails framework encourages RESTful design of your applications, which means you'll be making a lot of "PATCH" and "DELETE" requests (besides "GET" and "POST"). However, most browsers*don't support* methods other than "GET" and "POST" when it comes to submitting forms.
Rails works around this issue by emulating other methods over POST with a hidden input named `"_method"`, which is set to reflect the desired method:

```
form_tag(search_path, method: "patch")
```

output:

```
<form accept-charset="UTF-8" action="/search" method="post">
  <input name="_method" type="hidden" value="patch" />
  <input name="utf8" type="hidden" value="&#x2713;" />
  <input name="authenticity_token" type="hidden" value="f755bb0ed134b76c432144748a6d4b7a7dd
/>
  ...
</form>
```

When parsing POSTed data, Rails will take into account the special `_method` parameter and acts as if the HTTP method was the one specified inside it ("PATCH" in this example).

3 Making Select Boxes with Ease

Select boxes in HTML require a significant amount of markup (one `OPTION` element for each option to choose from), therefore it makes the most sense for them to be dynamically generated.
Here is what the markup might look like:

```
<select name="city_id" id="city_id">
  <option value="1">Lisbon</option>
  <option value="2">Madrid</option>
  ...
  <option value="12">Berlin</option>
</select>
```

Here you have a list of cities whose names are presented to the user. Internally the application only wants to handle their IDs so they are used as the options' value attribute. Let's see how Rails can help out here.

3.1 The Select and Option Tags

The most generic helper is `select_tag`, which - as the name implies - simply generates the `SELECT`tag that encapsulates an options string:

```
<%= select_tag(:city_id, '<option value="1">Lisbon</option>...') %>
```

This is a start, but it doesn't dynamically create the option tags. You can generate option tags with the`options_for_select` helper:

```
<%= options_for_select([['Lisbon', 1], ['Madrid', 2], ...]) %>

output:

<option value="1">Lisbon</option>
<option value="2">Madrid</option>
...
```

The first argument to `options_for_select` is a nested array where each element has two elements: option text (city name) and option value (city id). The option value is what will be submitted to your controller. Often this will be the id of a corresponding database object but this does not have to be the case.
Knowing this, you can combine `select_tag` and `options_for_select` to achieve the desired, complete markup: